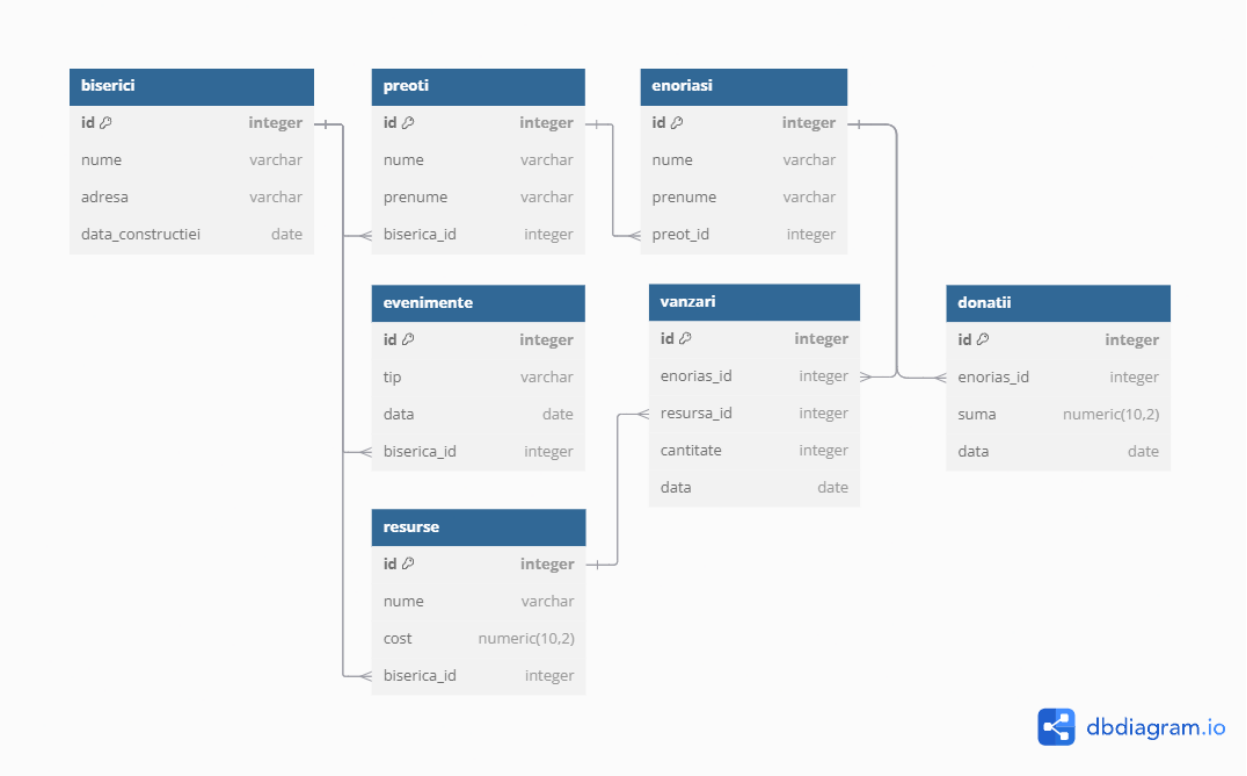
Descrierea temei

1. Descrierea bazei de date

O platformă de gestionare pentru biserici care va administra membrii, evenimentele, donațiile, vânzări de resurse și activități ale comunității.

2.1. Diagrama bazei de date



2.2. Structura tabelelor

Tabelul **Biserici** stochează detalii despre biserici, precum numele, adresa, data construcției și tipul bisericii (ex. Ortodoxă, Catolică etc.). Fiecare biserică are un identificator unic generat automat, denumit **IDBiserica**.

Tabelul **Preoti** conține informații despre preoți, incluzând numele, prenumele și referința către biserica unde activează. Fiecare preot are un identificator unic, **IDPreot**, iar **IDBiserica** leagă preotul de biserica sa.

Tabelul **Enoriasi** păstrează informații despre enoriași, cum ar fi numele, prenumele și referința către preotul care îi păstorește. Fiecare enoriaș este identificat prin **IDEnorias**, iar legătura cu preotul se face prin **IDPreot**.

Tabelul **Donatii** înregistrează donațiile făcute de enoriași. Fiecare donație este identificată prin **IDDonatie** și conține informații despre enoriașul donator, suma donată și data donației. Suma trebuie să fie întotdeauna pozitivă. Este legat de enoriașul donator prin **IDEnorias**.

Tabelul **Evenimente** conține informații despre activitățile organizate de biserici, cum ar fi slujbe, botezuri sau cununii. Fiecare eveniment are un identificator unic, IDEveniment, și este legat de biserica unde a avut loc prin **IDBiserica**.

Tabelul **Resurse** stochează informații despre materialele disponibile în biserici, precum numele resursei și costul acesteia. Fiecare resursă este identificată prin **IDResursa** și legată de biserică prin **IDBiserica**.

Tabelul **Vanzari** înregistrează vânzările de resurse către enoriași. Fiecare vânzare are un identificator unic, **IDVanzare**, și include detalii despre enoriașul cumpărător, resursa vândută, cantitatea și data vânzării. Cantitatea trebuie să fie întotdeauna pozitivă. Este legat de tabelul Resurse prin **IDResursa** și de tabelul Enoriasi prin **IDEnorias**.

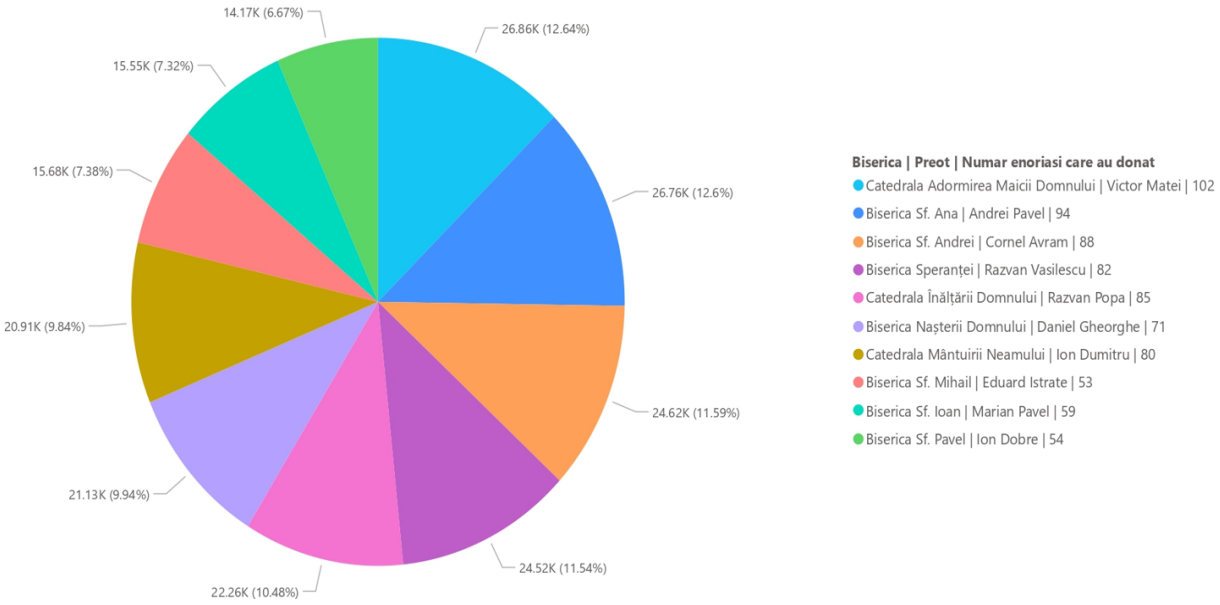
2.3. Descrierea constrângerilor de integritate

Baza de date utilizează chei primare pentru a asigura unicitatea fiecărei înregistrări, cum ar fi **IDBiserica**, **IDPreot**, **IDEnorias** etc. Cheile externe (străine) creează legături între tabele, asigurând consistența datelor. De exemplu, **IDBiserica** din tabelul **Preoti** se referă la **IDBiserica** din tabelul **Biserici**, iar **IDPreot** din tabelul **Enoriasi** se referă la **IDPreot** din tabelul **Preoti**.

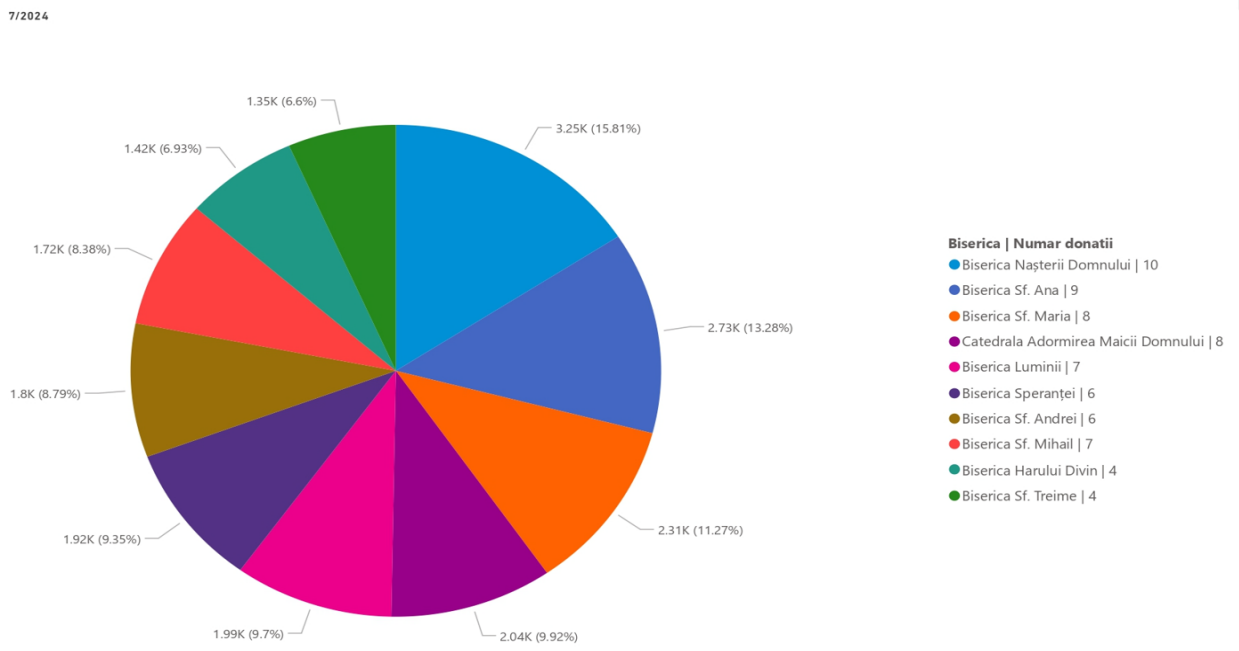
Constrângerile de tip **CHECK** garantează validitatea datelor, cum ar fi sumele pozitive pentru donații și costurile resurselor, sau cantitățile pozitive pentru vânzări. De asemenea, anumite coloane, cum ar fi data donației sau data vânzării, au valori implicite setate la data curentă pentru a simplifica introducerea datelor.

2.4. Descrierea procedurilor și funcțiilor

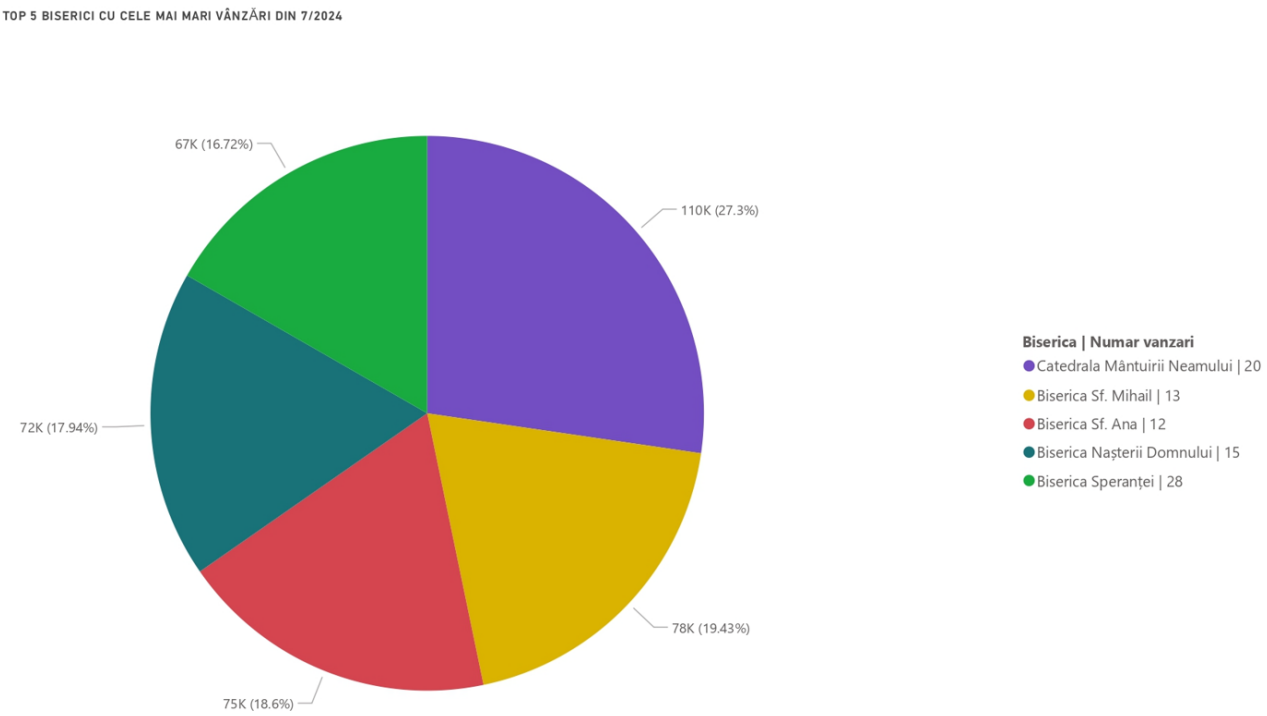
Procedura **raport_top10_preoti** (Complexitate: 4) generează un raport cu primii 10 preoți care au cei mai mulți enoriași donatori, împreună cu suma totală donată. Procedura combină datele din tabelele **Biserici**, **Preoti**, **Enoriasi** și **Donatii**. În cadrul procedurii, se grupează informațiile după biserică, numele și prenumele preotului, și se calculează numărul total de enoriași și suma totală donată. Rezultatele sunt sortate descrescător după numărul de enoriași, iar primele 10 intrări sunt afișate.



Procedura **raport_biserica_cu_cele_mai_mari_donatii_din_luna_cu_cele_mai_multe_evenimente** (Complexitate: 7) identifică top 10 al bisericilor cu cele mai mari donații în luna cu cele mai multe evenimente. În primul rând, se determină luna cu cele mai multe evenimente folosind tabelul **Evenimente**. Apoi, se calculează totalul donațiilor pentru fiecare biserică în acea lună, utilizând datele din tabelele **Donatii**, **Enoriasi**, **Preoti** și **Biserici**. Apoi se combina cele doua interogari pentru a afla topul bisericilor din luna cu cele mai multe evenimente, luandu-se bisericile cu 0 < Rnk <= 10.

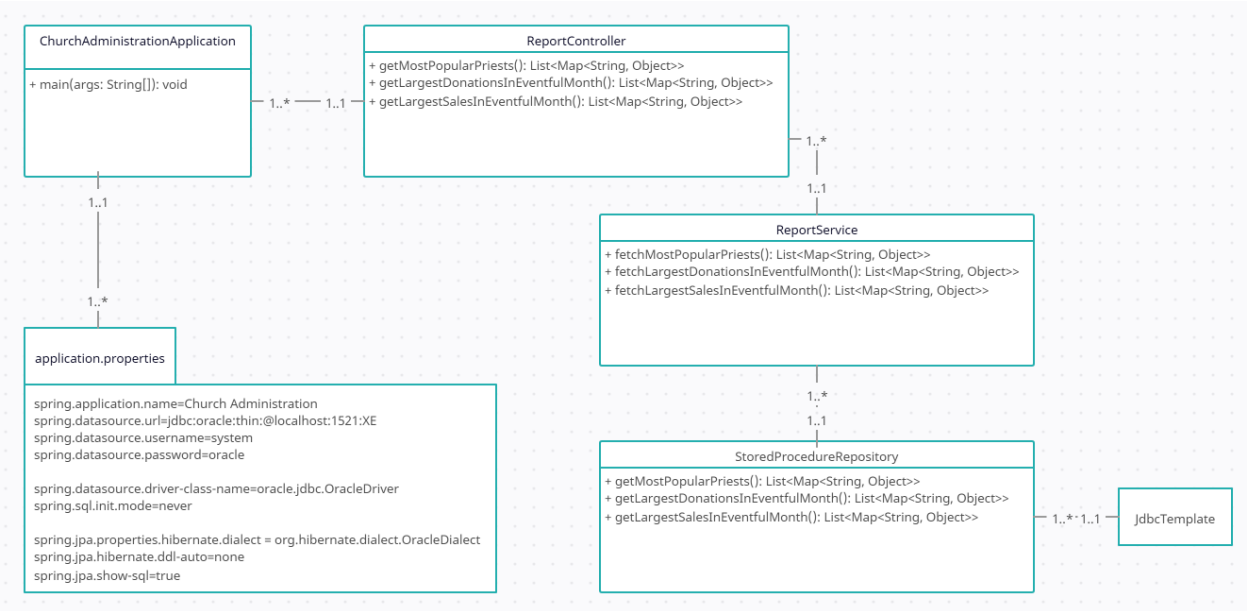


Procedura **raport_biserica_cu_cele_mai_mari_vanzari_din_luna_cu_cele_mai_multe_evenimente** (Complexitate: 8) oferă un top 5 al bisericilor cu cele mai mari vânzări în luna cu cele mai multe evenimente. Similar procedurii anterioare, se identifică mai întâi luna cu cele mai multe evenimente. Apoi, se calculează vânzările totale pentru fiecare biserică, folosind informațiile din tabelele **Vanzari**, **Enoriasi**, **Preoti**, **Biserici** și **Resurse**. Vânzările totale sunt calculate prin înmulțirea cantității vândute cu costul resursei. Rezultatele sunt sortate descrescător, iar primele 5 biserici sunt afișate, folosind la fel 0 < Rnk <= 5.



2. Descrierea aplicației

2.1. Diagrama de clase



2.2. Structura claselor

ChurchAdministrationApplication este punctul de intrare al aplicației și inițializează funcționarea acesteia prin metoda main.

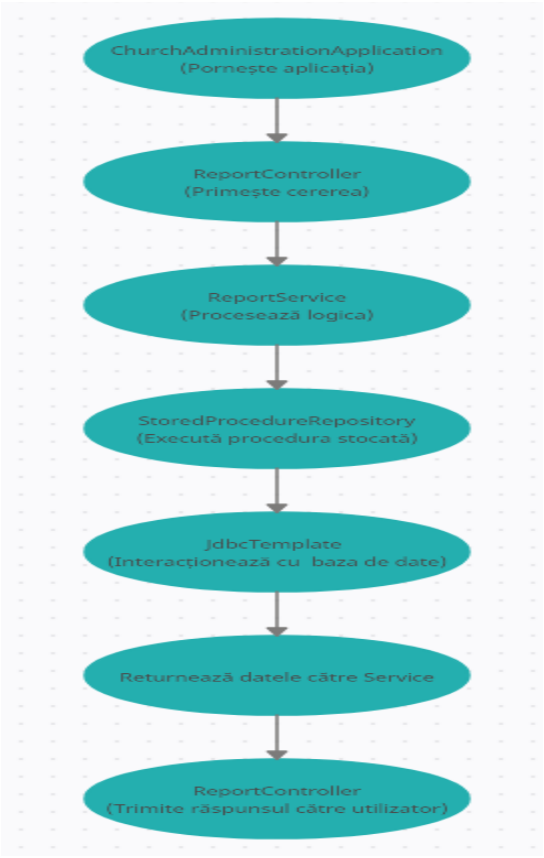
ReportController expune API-urile pentru generarea rapoartelor, incluzând metodele pentru obținerea preoților cei mai populari, donațiilor din luna cu cele mai multe evenimente și vânzărilor din aceeași perioadă.

ReportService implementează logica de business necesară pentru a prelua datele din repository și oferă metode pentru a obține informațiile despre preoți, donații și vânzări.

StoredProcedureRepository se ocupă de executarea procedurilor stocate din baza de date, preluând informațiile necesare pentru rapoarte.

JdbcTemplate este componenta utilizată pentru interacțiunea directă cu baza de date, fiind responsabilă de executarea query-urilor și procedurilor stocate.

2.3. Diagrama de stări și fluxul de lucru (workflow) pentru aplicație



2.4. Prezentarea modului în care se face conexiunea cu baza de date

Conexiunea cu baza de date este realizată prin Spring JDBC folosind JdbcTemplate. Parametrii de conexiune, precum URL-ul bazei de date, utilizatorul și parola, sunt definiți în fișierul **application.properties**. Spring Boot configurează automat un DataSource, iar JdbcTemplate este injectat în repository-uri pentru a executa procedurile stocate și interogările SQL. Conexiunea este gestionată automat printr-un pool de conexiuni (ex. HikariCP), fără a fi necesară intervenția manuală pentru deschiderea sau închiderea conexiunii.

3. Capturi de ecran pentru interfețe și rapoarte



```sql
CREATE OR REPLACE PROCEDURE raport_biserica_cu_cele_mai_mari_vanzari_din_luna_cu_cele_mai_multe_evenimente (
    p_cursor OUT SYS_REFCURSOR
) AS
BEGIN
    OPEN p_cursor FOR
        WITH EvenimenteLuna AS (
            SELECT
                TO_CHAR(E.Data, 'MM') AS Luna,
                TO_CHAR(E.Data, 'YYYY') AS An,
                COUNT(E.IDEveniment) AS NumarEvenimente
            FROM
                Evenimente E
            GROUP BY
                TO_CHAR(E.Data, 'MM'), TO_CHAR(E.Data, 'YYYY') -- 1
            ORDER BY
                NumarEvenimente DESC
            FETCH FIRST 1 ROWS ONLY
        ),
        VanzariLuna AS (
            SELECT
                TO_CHAR(V.Data, 'MM') AS Luna,
                TO_CHAR(V.Data, 'YYYY') AS An,
                B.IDBiserica,
                B.Nume AS Biserica,
                SUM(V.Cantitate * R.Cost) AS SumaTotala,
                COUNT(V.IDVanzare) AS NumarVanzari
            FROM
                Vanzari V
            JOIN
                Enoriasi E ON V.IDEnorias = E.IDEnorias -- 2
            JOIN
                Preoti P ON E.IDPreot = P.IDPreot -- 3
            JOIN
                Biserici B ON P.IDBiserica = B.IDBiserica -- 4
            JOIN
                Resurse R ON V.IDResursa = R.IDResursa -- 5
            GROUP BY
                TO_CHAR(V.Data, 'MM'), TO_CHAR(V.Data, 'YYYY'), B.IDBiserica, B.Nume -- 6
        ),
        MaxVanzariLuna AS (
            SELECT
                VL.Luna,
                VL.An,
                VL.Biserica,
                VL.SumaTotala,
                VL.NumarVanzari,
                RANK() OVER (PARTITION BY VL.Luna, VL.An ORDER BY VL.SumaTotala DESC) AS Rnk
            FROM
                VanzariLuna VL
            JOIN
                EvenimenteLuna EL ON VL.Luna = EL.Luna AND VL.An = EL.An -- 7
        )
        SELECT
            Luna,
            An,
            Biserica,
            SumaTotala,
            NumarVanzari
        FROM
            MaxVanzariLuna
        WHERE
            Rnk > 0 AND Rnk <= 5 -- 8
        ORDER BY
            An, Luna, SumaTotala DESC;
END;
/
```



```sql
CREATE OR REPLACE PROCEDURE raport_biserica_cu_cele_mai_mari_donatii_din_luna_cu_cele_mai_multe_evenimente (
    p_cursor OUT SYS_REFCURSOR
) AS
BEGIN
    OPEN p_cursor FOR
        WITH EvenimenteLuna AS (
            SELECT
                TO_CHAR(E.Data, 'MM') AS Luna,
                TO_CHAR(E.Data, 'YYYY') AS An,
                COUNT(E.IDEveniment) AS NumarEvenimente
            FROM
                Evenimente E
            GROUP BY
                TO_CHAR(E.Data, 'MM'), TO_CHAR(E.Data, 'YYYY') -- 1
            ORDER BY
                NumarEvenimente DESC
            FETCH FIRST 1 ROWS ONLY
        ),
        DonatiiLuna AS (
            SELECT
                TO_CHAR(D.Data, 'MM') AS Luna,
                TO_CHAR(D.Data, 'YYYY') AS An,
                B.IDBiserica,
                B.Nume AS Biserica,
                SUM(D.Suma) AS SumaTotala,
                COUNT(D.IDDonatie) AS NumarDonatii
            FROM
                Donatii D
            JOIN
                Enoriasi E ON D.IDEnorias = E.IDEnorias -- 2
            JOIN
                Preoti P ON E.IDPreot = P.IDPreot -- 3
            JOIN
                Biserici B ON P.IDBiserica = B.IDBiserica -- 4
            GROUP BY
                TO_CHAR(D.Data, 'MM'), TO_CHAR(D.Data, 'YYYY'), B.IDBiserica, B.Nume -- 5
        ),
        MaxDonatiiLuna AS (
            SELECT
                DL.Luna,
                DL.An,
                DL.Biserica,
                DL.SumaTotala,
                DL.NumarDonatii,
                RANK() OVER (PARTITION BY DL.Luna, DL.An ORDER BY DL.SumaTotala DESC) AS Rnk
            FROM
                DonatiiLuna DL
            JOIN
                EvenimenteLuna EL ON DL.Luna = EL.Luna AND DL.An = EL.An -- 6
        )
        SELECT
            Luna,
            An,
            Biserica,
            SumaTotala,
            NumarDonatii
        FROM
            MaxDonatiiLuna
        WHERE
            Rnk > 0 AND Rnk <= 10 -- 7
        ORDER BY
            An, Luna, SumaTotala DESC;
END;
/
```



```sql
-- Top 10 preoti care au cei mai multi enoriași care au donat
CREATE OR REPLACE PROCEDURE raport_top10_preoti (
    p_cursor OUT SYS_REFCURSOR
) AS
BEGIN
    OPEN p_cursor FOR
        SELECT
            B.Nume AS Biserica,
            P.Nume,
            P.Prenume,
            COUNT(E.IDEnorias) AS NumarEnoriasi,
            SUM(D.Suma) AS SumaTotalaDonata
        FROM
            Biserici B
        JOIN
            Preoti P ON B.IDBiserica = P.IDBiserica -- 1
        JOIN
            Enoriasi E ON P.IDPreot = E.IDPreot -- 2
        JOIN
            Donatii D ON E.IDEnorias = D.IDEnorias -- 3
        GROUP BY
            B.Nume, P.Nume, P.Prenume -- 4
        ORDER BY
            COUNT(E.IDEnorias) DESC
        FETCH FIRST 10 ROWS ONLY;
END;
/
```

4. Concluzii

Proiectul a fost interesant, am apreciat că a trebuit să învăț să fac lucruri noi față de laborator, însă trebuia să fie mai mult ajutor oferit din partea asistenților și cerința mai bine explicată. Dar per total am reușit să înțeleg mai bine cum se crează o bază de date, cum adaugi batch-uri mari de date și cum o gestionezi cu ajutorul unei aplicații.

5. Bibliografie

Laboratoare:

https://ocw.cs.pub.ro/courses/bdd/laboratoare/01-06

PL/SQL:

https://ocw.cs.pub.ro/courses/bdd/resurse/docker

Spring:
https://start.spring.io/
Spring cu PL/SQL:
https://docs.spring.io/spring-data/data-jdbc/old-docs/2.0.x/reference/html/orcl.datatypes.html